

25 AÑOS



Ingeniería de las
Telecomunicaciones

Firebase - Cloud Firestore

1TEL05 - Servicios y Aplicaciones para IoT

Prof. Oscar Díaz

Clase 9.2



—
Ingeniería de las
Telecomunicaciones

Base de datos

Firestore ofrece dos soluciones de bases de datos no relacionales (**NoSQL**) en la nube y accesibles para los clientes que admiten sincronización en tiempo real:



Cloud Firestore, base de datos más reciente de Firebase para el desarrollo de apps para dispositivos móviles. Se pueden realizar consultas más ricas y rápidas, y el escalamiento se ajusta a un nivel más alto que Realtime Database.

→ Tiene librerías para **iOS, Android, Web, Flutter, Java, Python, NodeJs...**



Realtime Database, base de datos original de Firebase. Es una solución eficiente y de baja latencia destinada a las apps para dispositivos móviles que necesitan estar sincronizados entre los clientes en tiempo real.

→ Tiene librerías para **iOS, Android, Web, C++ y Unity**

Cloud Firestore

- Base de datos NoSQL, permite almacenar y sincronizar datos entre los dispositivos conectados en tiempo real.
- Tiene librerías para:  además de REST y RPC APIs
- Esquema Serverless
- Acceso a los datos mediante:
 - Firebase auth.
 - Acceso abierto (para pruebas)
 - Server to Server
- Documentación: <https://firebase.google.com/docs/firestore?hl=es-419>

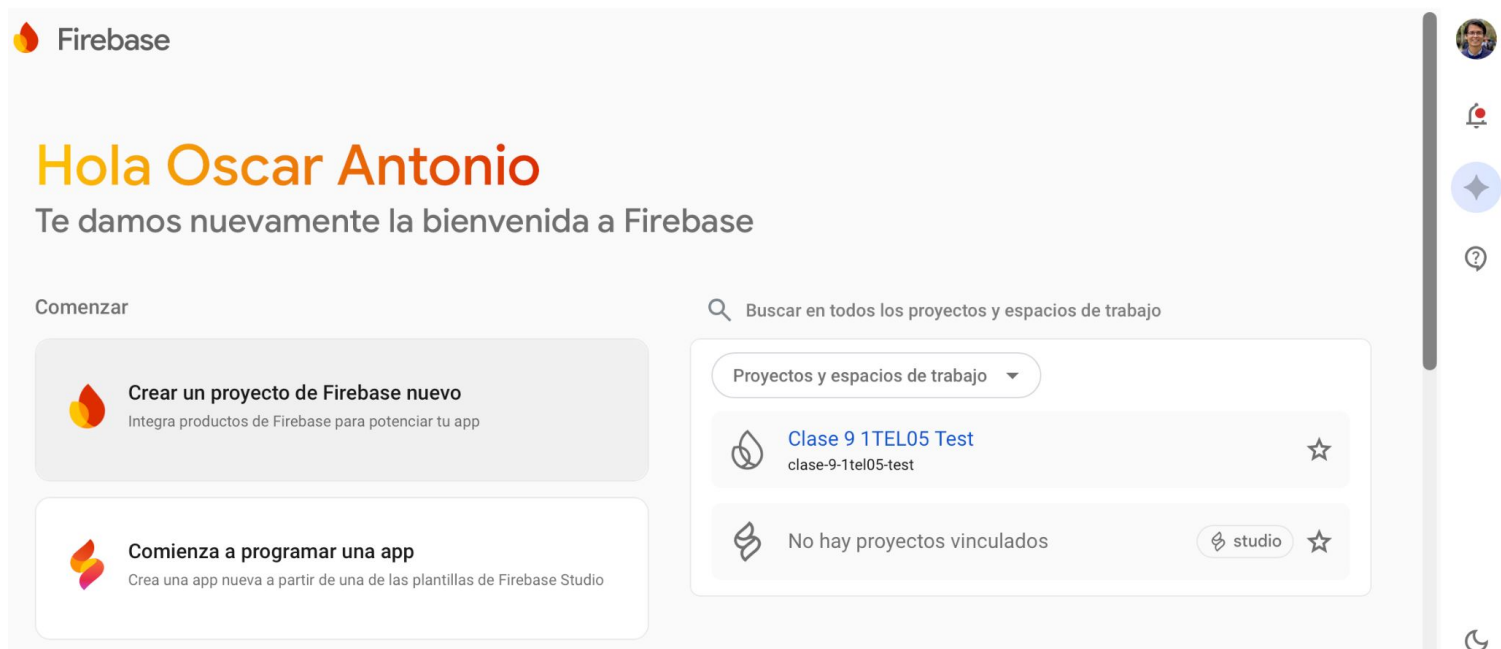


Proyecto en Google Console

Antes de empezar a enviar y recibir información, es necesario crear un proyecto en google.

Entre a <https://console.firebase.google.com/>, use su correo @pucp.edu.pe o @pucp.pe

Seleccione “Agregar proyecto”.



Crear proyecto - firebase console

× Crear un proyecto

Comencemos con el nombre de tu proyecto[?]

Nombre del proyecto

Clase 9 1TEL05

 clase-9-1tel05-c5381

 pucp.pe

☒ Únete al [Programa para desarrolladores de Google](#) para enriquecer tu experiencia como desarrollador con acceso a asistencia de IA, recursos de aprendizaje, insignias de perfil y mucho más.

¿Ya tienes un proyecto de Google Cloud?
[Agregar Firebase al proyecto de Google Cloud](#)

Continuar

Asegúrese de seleccionar **pucp.pe** o **pucp.edu.pe**

Crear proyecto - firebase console

Google Analytics para tu proyecto de Firebase

Google Analytics es una solución de analítica ilimitada y gratuita que permite usar la segmentación, los informes y otras funciones en Firebase Crashlytics, Cloud Messaging, In-App Messaging, Remote Config, A/B Testing y Cloud Functions.

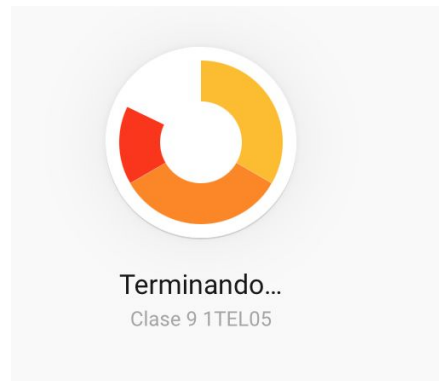
Google Analytics habilita las siguientes funciones:

× Pruebas A/B ⓘ	× Registros de rutas de navegación en Crashlytics ⓘ
× Segmentación de usuarios y orientación a ellos en los productos de Firebase ⓘ	× Activadores de Cloud Functions basados en eventos ⓘ
	× Informes ilimitados y gratuitos ⓘ

☒ Habilitar Google Analytics para este proyecto
Recomendado

[Anterior](#) [Crear proyecto](#)

Crear proyecto - firebase console



Firebase

 **Firebase**

[Descripción gene...](#) 

Novedades

 AI Logic **NUEVO**

Categorías de producto

- Compilación ▾
- Ejecución ▾
- Analytics ▾
- IA ▾

Clase 9 1TEL05 ▾

 Prueba una app de agente de barista potenciada por Firebase AI Logic. [Probar la app de ejemplo](#) 

Clase 9 1TEL05 **Plan Spark**

[+ Agregar app](#)

Hola Oscar Antonio

Te damos la bienvenida a tu proyecto de Firebase

Gemini te puede sugerir servicios y soluciones de Firebase para tu app

Próximos pasos con Gemini [Ver los documentos](#) 

Configurando Firebase en Android



Ingeniería de las
Telecomunicaciones

Requisitos

- API Level ≥ 19 (Android 4.4 KitKat)
- compileSdkVersion 28 o una versión posterior

→ <https://firebase.google.com/docs/android/setup?hl=es>

Registrar la app



Firebase

Clase 9 1TEL05 ▼

Prueba una app de agente de barista potenciada por Firebase AI Logic. [Probar la app de ejemplo](#) ✕

Clase 9 1TEL05 Plan Spark

✕      Seleccionar plataforma

Hola, Oscar Antonio

Te damos la bienvenida a tu proyecto de Firebase

Gemini te puede sugerir servicios y soluciones de Firebase para tu app

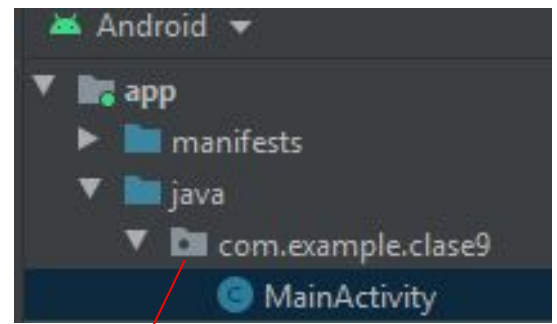
Próximos pasos con Gemini [Ver los documentos](#)

Registrar el ID de la APP

Ingrese el ID completo de su aplicación. En este caso se ha creado un proyecto:

→ *com.example.clase9*

*** Este valor no se puede cambiar después**



× **Agregar Firebase a tu app para Android**

1 **Registrar app**

Nombre del paquete de Android ⓘ

Sobrenombre de la app(opcional) ⓘ

2 Descargar y, luego, agregar el archivo de configuración

3 Agregar el SDK de Firebase

4 Próximos pasos

Estos campos son opcionales

Descarga el archivo de configuración

Descargue el archivo de configuración y coloquelo donde indica

✓ Registrar app
Nombre del paquete de Android: com.example.clase9

2 Descargar y, luego, agregar el archivo de configuración
Android Studio a continuación | [Unity](#) [C++](#)

Descargar google-services.json

Cambia a la vista **Proyecto** en Android Studio para ver el directorio raíz de tu proyecto.

Mueve el archivo descargado `google-services.json` al directorio raíz de tu módulo (nivel de app).


google-services.json

Siguiente

Project

MyApplication [My Application]

gradle

idea

app

libs

src

.gitignore

build.gradle.kts

google-services.json

proguard-rules.pro

gradle

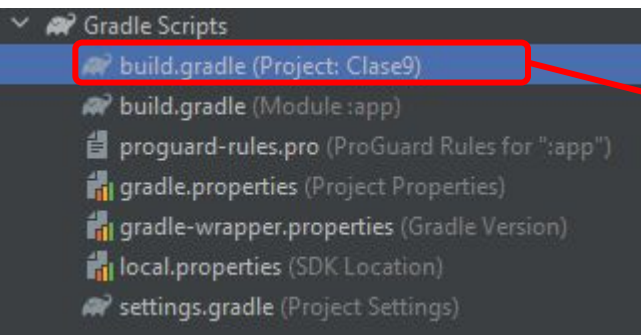
3 Agregar el SDK de Firebase

4 Próximos pasos

Agregar el SDK de Google Services

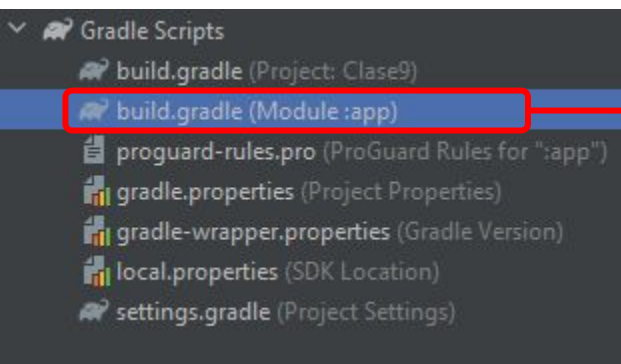
Realice las configuraciones de gradle indicadas:

→ build.gradle (**Project:** <proyecto>)



```
plugins {
    id 'com.android.application' version '8.6.0' apply false
    id 'com.google.gms.google-services' version '4.4.2' apply false
}
```

→ build.gradle (**Module:** app)



```
plugins {
    id 'com.android.application'
    id 'com.google.gms.google-services'
}

android {
    namespace 'com.example.clase9'
    compileSdk 33
}
```

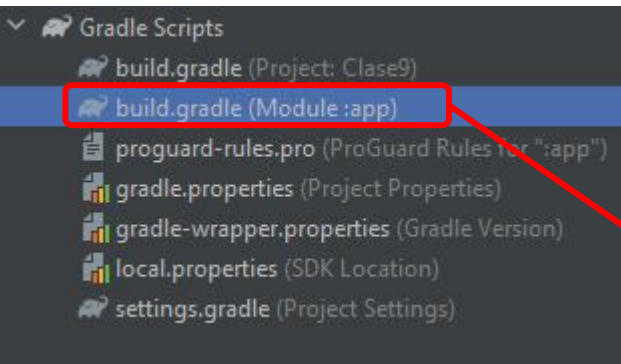
Agregando el Firebase SDK



—
Ingeniería de las
Telecomunicaciones

Agregar el SDK base de Firebase

→ build.gradle (**Module: app**)



```
dependencies {  
  
    implementation 'androidx.appcompat:appcompat:1.6.1'  
    implementation 'com.google.android.material:material:1.9.0'  
    implementation 'androidx.constraintlayout:constraintlayout:2.1.4'  
    testImplementation 'junit:junit:4.13.2'  
    androidTestImplementation 'androidx.test.ext:junit:1.1.5'  
    androidTestImplementation 'androidx.test.espresso:espresso-core:3.5.1'  
  
    implementation platform('com.google.firebase:firebase-bom:32.4.1')  
}
```

Verifique aquí la última versión:

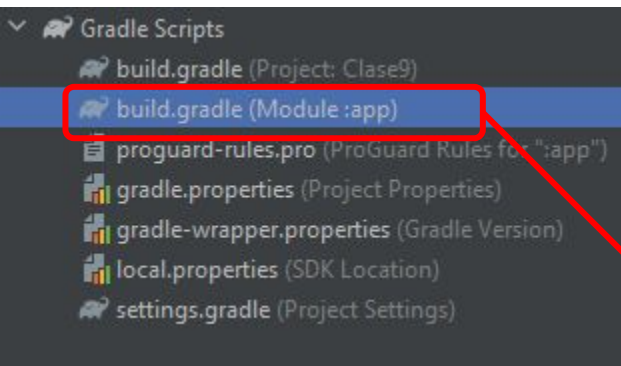
<https://mvnrepository.com/artifact/com.google.firebase/firebase-bom>

Agregar el SDK de Cloud Firestore

→ build.gradle (**Module: app**)



Como usamos firebase-bom, ya no es necesario indicar la versión.



```
dependencies {  
  
    implementation 'androidx.appcompat:appcompat:1.6.1'  
    implementation 'com.google.android.material:material:1.9.0'  
    implementation 'androidx.constraintlayout:constraintlayout:2.1.4'  
    testImplementation 'junit:junit:4.13.2'  
    androidTestImplementation 'androidx.test.ext:junit:1.1.5'  
    androidTestImplementation 'androidx.test.espresso:espresso-core:3.5.1'  
  
    implementation platform('com.google.firebase:firebase-bom:32.4.1')  
    implementation 'com.google.firebase:firebase-firestore'  
}
```

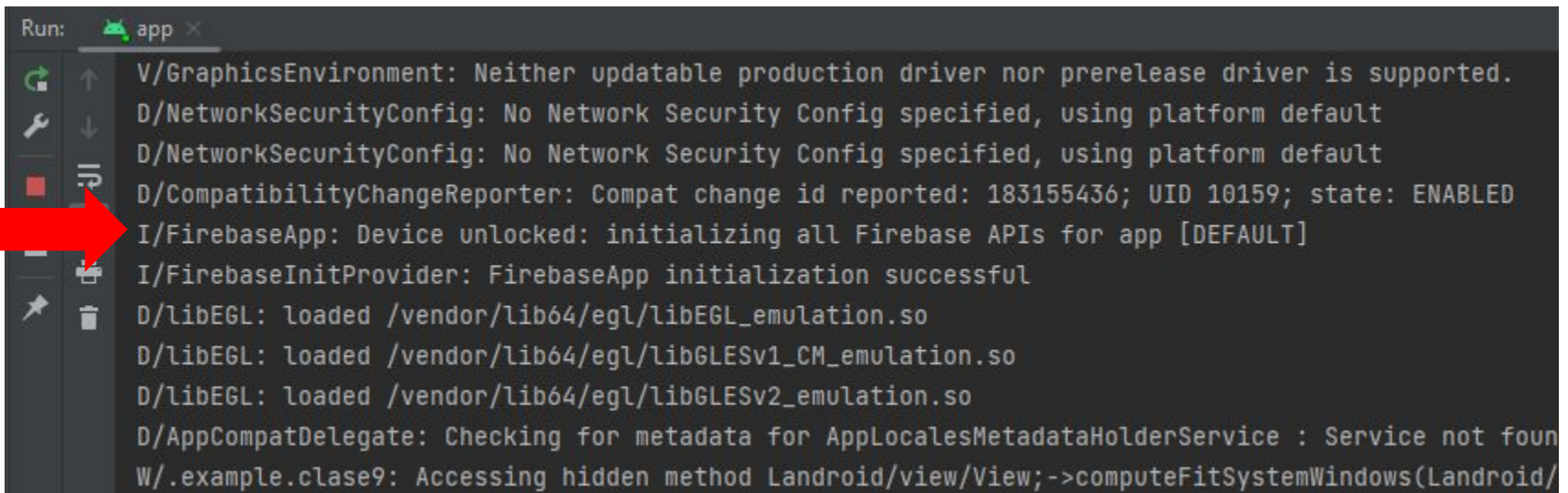
Si desea otra librería de Firebase, la puede ubicar en:

→ <https://firebase.google.com/docs/android/setup?hl=es#available-libraries>

Listo

El SDK de Firebase usará el archivo de configuración para autenticarse.

Corra el proyecto (aún si está vacío) para probar y ver que no exista error.



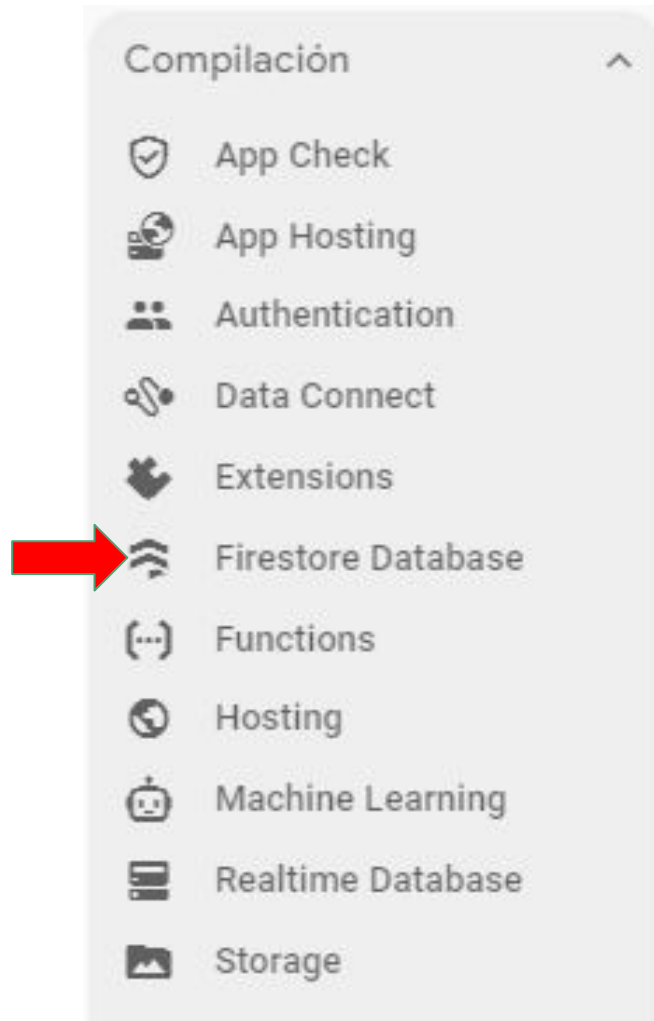
```
Run: app x
V/GraphicsEnvironment: Neither updatable production driver nor prerelease driver is supported.
D/NetworkSecurityConfig: No Network Security Config specified, using platform default
D/NetworkSecurityConfig: No Network Security Config specified, using platform default
D/CompatibilityChangeReporter: Compat change id reported: 183155436; UID 10159; state: ENABLED
I/FirebaseApp: Device unlocked: initializing all Firebase APIs for app [DEFAULT]
I/FirebaseInitProvider: FirebaseApp initialization successful
D/libEGL: loaded /vendor/lib64/egl/libEGL_emulation.so
D/libEGL: loaded /vendor/lib64/egl/libGLESv1_CM_emulation.so
D/libEGL: loaded /vendor/lib64/egl/libGLESv2_emulation.so
D/AppCompatDelegate: Checking for metadata for AppLocalesMetadataHolderService : Service not found
W/.example.clase9: Accessing hidden method Landroid/view/View;->computeFitSystemWindows(Landroid/view/View;[Landroid/view/View;)
```

Creando la base de datos Cloud Firestore



Ingeniería de las
Telecomunicaciones

Crear una base de datos Cloud Firestore



Crear una base de datos Cloud Firestore

Clase 9 1TEL05 ▼

Cloud Firestore

Actualizaciones en tiempo real, consultas potentes, escalado automático y compatibilidad con MongoDB

Crear base de datos

◆ Preguntarle a Gemini



Crear una base de datos Cloud Firestore

× Crea una base de datos

1 Selecccionar edición

☒ Edición Standard
Motor de consultas simples con indexación automática. Para documentos de hasta 1 MiB.

☐ Edición Enterprise
Motor de consultas avanzadas con compatibilidad con MongoDB. Para documentos de hasta 4 MiB. Solo admite controladores y herramientas de MongoDB.

¿No sabes cuál es la edición ideal para ti? [Comparar ediciones](#)

Siguiente

2 ID y ubicación de la base de datos

Crear una base de datos Cloud Firestore

× Crea una base de datos

1

Seleccionar edición

2

ID y ubicación de la base de datos

3

Configurar

ID de la base de datos

(default)

Ubicación

nam5 (United States) ▼

ⓘ

La configuración de ubicación es el lugar donde se almacenarán tus datos de Cloud Firestore

!

No podrás cambiar la ubicación después de configurarla.

Más información [↗](#)

Siguiente

Crear una base de datos Cloud Firestore

×

Crea una base de datos

Seleccionar edición

ID y ubicación de la base de datos

3

Configurar

Después de definir la estructura de tus datos, debes crear reglas para protegerlos. [Más información](#)

☐

Iniciar en modo de producción

Tus datos son privados de forma predeterminada. El acceso de lectura/escritura de los clientes solo se otorgará como se indica en tus reglas de seguridad.

☒

Comenzar en modo de prueba

Para permitir una configuración rápida, los datos se abren de forma predeterminada. Sin embargo, debes actualizar las reglas de seguridad en un plazo de 30 días para habilitar el acceso de lectura/escritura a largo plazo para los clientes.

```

rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if
        request.time < timestamp.date(2025, 11, 17);
    }
  }
}

```

!

Las reglas de seguridad predeterminadas del modo de prueba permiten que cualquier persona con acceso a la referencia de tu base de datos pueda ver, editar y borrar todos los datos de esta durante los siguientes 30 días

Cancelar

Crear

Base de datos Cloud Firestore

Clase 9 1TEL05 ▾

Cloud Firestore >

Base de datos

Agregar base de datos

Preguntarle a Gemini cómo comenzar a usar Firestore

Datos

Reglas

Índices

Recuperación ante desastres

Uso

Extensions

Protege tus recursos de Cloud Firestore contra los abusos, como fraudes de facturación o phishing.

Configurar la Verificación de aplicaciones

Vista del panel

Compilador de consultas

Más funciones en Google Cloud

(default)	
+ Iniciar colección	

Estructura de datos en Cloud Firestore

Almacena los datos en **documentos** que contienen campos a los cuales se les asignan valores. Estos documentos se almacenan en **colecciones**, que son contenedores para documentos y sobre los cuales se pueden crear consultas.

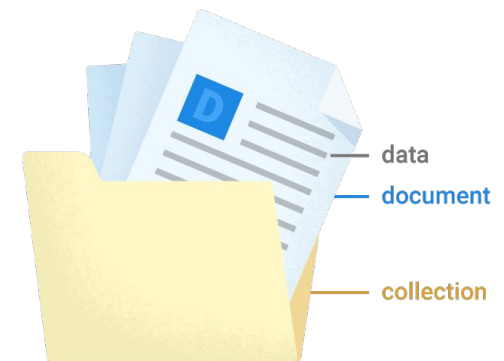
Los documentos admiten muchos tipos de datos diferentes, desde simples cadenas y números hasta objetos complejos anidados. También puede crear subcolecciones dentro de documentos y crear estructuras de datos jerárquicas que se escalan a medida que crece su base de datos.

Más información en:

<https://firebase.google.com/docs/firestore/data-model>

Así mismo, existen diferentes formas de ordenar su data:

<https://firebase.google.com/docs/firestore/manage-data/structure-data>



Guardar información en Realtime Database



Ingeniería de las
Telecomunicaciones

Tipos de datos en firebase

Los tipos de datos soportados en firebase son:

<https://firebase.google.com/docs/firestore/manage-data/data-types>

1. Null values
2. Boolean values
3. Integer and floating-point values, sorted in numerical order
4. Date values
5. Text string values
6. Byte values
7. Cloud Firestore references
8. Geographical point values
9. Array values
10. Map values

Instancia de la BD

Para acceder a Realtime Database debemos obtener la instancia de conexión.

```
Firestore db;  
  
@Override  
protected void onCreate(Bundle savedInstanceState) {  
    super.onCreate(savedInstanceState);  
    binding = ActivityMainBinding.inflate(getLayoutInflater());  
    setContentView(binding.getRoot());  
  
    db = FirebaseFirestore.getInstance();  
}
```

Obtiene una instancia de su base de datos usando su archivo de credenciales

Objeto a guardar

Existen dos formas para organizar la data que sea guardar:

- Utiliza HashMap
- Crea una clase con la estructura de la data

```
public class UsuarioDto {  
  
    private String nombre;  
    private String correo;  
    private String dni;  
}
```

Guardar información - **set()**

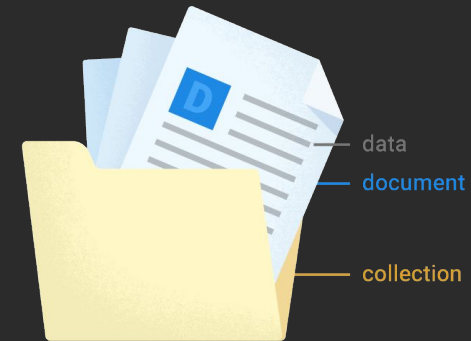
Se usa el método `set()` para guardar la información. Sin embargo, no puede guardar un usuario directamente, pues este es un “documento” y estos existen dentro de colecciones. Primero se define la colección, luego el identificador (id) y dentro el documento.

```

db = FirebaseFirestore.getInstance();

UsuarioDto usuario = new UsuarioDto();
usuario.setNombre( "Juan" );
usuario.setCorreo( "juan.perez@pucp.edu.pe" );
usuario.setDni( "12345678" );

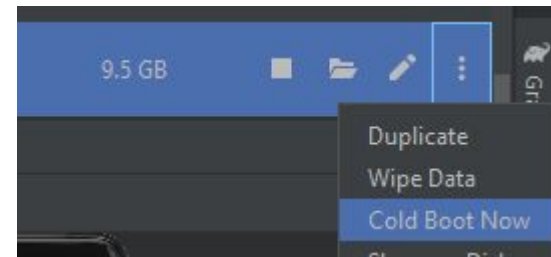
db.collection( "usuarios" )
    .document( "12345678" )
    .set( usuario )
    .addOnSuccessListener( unused -> {
        Log.d( "msg-test", "Data guardada exitosamente" );
    } )
    .addOnFailureListener( e -> e.printStackTrace() );
  
```



Probar la aplicación - errores

Si no puede guardar, verifique que tiene internet en el dispositivo y que tenga los permisos necesarios.

Para reiniciar el dispositivo, apáguelo y cold boot.



Resultado

Collection → Document → Data

Vista del panel

Compilador de consultas

🏠

>

usuarios

>

12345678

Más funciones en Google Cloud

📁 clase-9-1tel05-188d9	📁 usuarios ☰ ⋮	📄 12345678 ☰ ⋮
+ Iniciar colección	+ Agregar documento	+ Iniciar colección
usuarios >	12345678 >	+ Agregar campo correo: "juan.perez@pucp.edu.pe" dni: "12345678" nombre: "Juan"

Reglas en Firebase

Por defecto Firebase bloquea todas las conexiones a una base de datos de aquellos clientes que no estén autenticados.

Esto se

Clase 9 1TEL05 ▾

Cloud Firestore > Base de datos

Base de datos

Agregar base de datos

Preguntarle a Gemini cómo comenzar a usar Firestore

Datos

Reglas

Índices

Recuperación ante desastres

Uso

Extensions

Desarrollar y realizar pruebas



Protege tus datos con reglas que definan quién tiene acceso a ellos y cómo se estructuran

[Ver los documentos](#)

```

1 rules_version = '2';
2
3 service cloud.firestore {
4   match /databases/{database}/documents {
5
6     // This rule allows anyone with your Firestore database reference to view, edit,
7     // and delete all data in your Firestore database. It is useful for getting
8     // started, but it is configured to expire after 30 days because it
9     // leaves your app open to attackers. At that time, all client
10    // requests to your Firestore database will be denied.
11    //
12    // Make sure to write security rules for your app before that time, or else
13    // all client requests to your Firestore database will be denied until you Update
14    // your rules
15    match /{document=**} {
16      allow read, write: if request.time < timestamp.date(2025, 11, 17);
17    }
18  }
19 }
```

Reglas en Firebase

Existen dos formas de permitir la lectura y escritura de información:

- Autenticarnos con Firebase auth (<https://firebase.google.com/docs/auth>)
- Permitir escritura y lectura por colección.

Usaremos el segundo enfoque. La siguiente clase se probará con Firebase auth.

→ También puede encontrar información en:

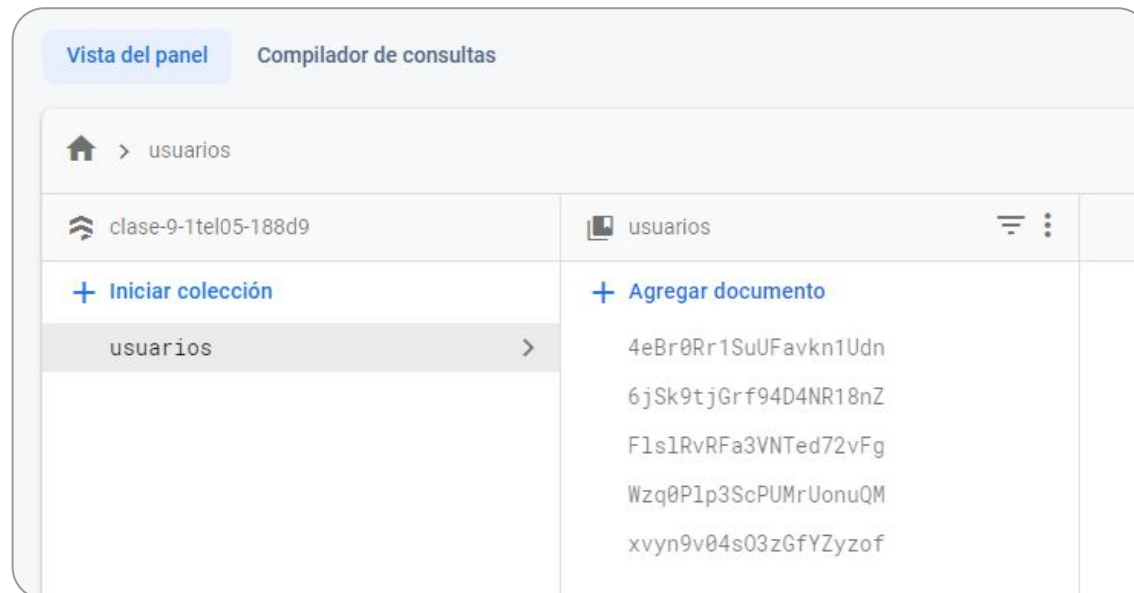
<https://firebase.google.com/docs/firestore/security/get-started>

Autogenerar ID.

También es posible dejar a Firebase que cree un ID por nosotros, de esta forma si se desea guardar una lista grande de datos, Firebase asignará a cada uno, un ID diferente.

Para esto, debe usar el método **add()** sobre la colección.

Por ejemplo, agregar usuarios con ID autogenerados.



Autogenerar ID.

Cada vez que se cree un usuario, este se creará dentro de <usuarios> con un ID autogenerado.

```
UsuarioDto usuario = new UsuarioDto();
usuario.setNombre("Juan");
usuario.setCorreo("juan.perez@pucp.edu.pe");
usuario.setDni("12345678");

db.collection("usuarios")
    .add(usuario)
    .addOnSuccessListener(unused -> {
        Log.d("msg-test", "Data guardada exitosamente");
    })
    .addOnFailureListener(e -> e.printStackTrace());
```

Esto es útil si nos importan los contenidos (de la lista) más que los ID de cada uno.

Leer datos



Ingeniería de las
Telecomunicaciones

Leer datos

Debido a que Cloud Firestore está orientado a obtener los datos en tiempo real, existen dos formas:

- **Usar Retrofit hacia el EndPoint** → no es recomendado porque se tendría que estar haciendo consultas cada cierto tiempo para ver si hay cambios en los datos. No sería tan “real-time”.
 - **Usar un Listener:**
 - **Por única vez:** Permite obtener la información una única vez, por ejemplo, cuando el usuario se conecta y no se esperan cambios en esa data.
 - **Siempre escuchando:** Crea un socket de conexión entre el dispositivo y firebase que le notifica cuando existe algún cambio.
- Para sacarle todo el provecho a Firebase, la idea es ir por un listener.

Para un documento



Ingeniería de las
Telecomunicaciones

Obtener un documento una vez

Para empezar a escuchar por cambios se utiliza el método **get()**.

En este ejemplo se obtiene el valor del documento con id: **4eBr0Rr1SuUFavkn1Udn**

```
db.collection("usuarios")
  .document("4eBr0Rr1SuUFavkn1Udn")
  .get()
  .addOnCompleteListener(task -> {
    if (task.isSuccessful()) {
      DocumentSnapshot document = task.getResult();
      if (document.exists()) {
        Log.d("msg-test", "DocumentSnapshot data: " + document.getData());
      } else {
        Log.d("nsg-test", "No such document");
      }
    } else {
      Log.d("msg-test", "get failed with ", task.getException());
    }
  });
```

Logcat:

```
D DocumentSnapshot data: {correo=juan.perez@pucp.edu.pe, nombre=Juan, dni=12345678}
```

Parsear la respuesta

La información que obtiene de Cloud Firestore, está en texto plano, puede utilizar una librería como Gson para parsear los resultados; o utilizar el parsear nativo de firebase **toObject()**.

```
db.collection("usuarios")
  .document("4eBr0Rr1SuUFavkn1Udn")
  .get()
  .addOnCompleteListener(task -> {
    if (task.isSuccessful()) {
      DocumentSnapshot document = task.getResult();

      if (document.exists()) {
        UsuarioDto usuario = document.toObject(UsuarioDto.class);

        if(usuario != null){
          Log.d("msg-test", "Nombre: " + usuario.getNombre());
          Log.d("msg-test", "Correo: " + usuario.getCorreo());
        }
      } else {
        Log.d("msg-test", "No such document");
      }
    } else {
      Log.d("msg-test", "get failed with ", task.getException());
    }
  });
```

Para una colección



Ingeniería de las
Telecomunicaciones

Obtener una colección una vez

Para empezar a escuchar por cambios se utiliza el método **get()** sobre la colección y luego se itera sobre la misma con **QueryDocumentSnapshot**

```
db.collection("usuarios")
    .get()
    .addOnCompleteListener(task -> {
        if (task.isSuccessful()) {
            for (QueryDocumentSnapshot document : task.getResult()) {
                UsuarioDto usuario = document.toObject(UsuarioDto.class);
                Log.d("msg-test", "Nombre: " + usuario.getNombre());
                Log.d("msg-test", "Correo: " + usuario.getCorreo());
            }
        } else {
            Log.d("msg-test", "Error getting documents: ", task.getException());
        }
    });
```

Obtener una colección y su cambios en tiempo real

Para empezar a escuchar por cambios se utiliza el método **addSnapshotListener()** pero en este caso, sobre la colección.

```
db.collection("usuarios")
    .addSnapshotListener((snapshot, error) -> {
        if (error != null) {
            Log.w("msg-test", "Listen failed.", error);
            return;
        }

        for (QueryDocumentSnapshot doc: snapshot) {
            UsuarioDto usuario = doc.toObject(UsuarioDto.class);
            Log.d("msg-test", "Nombre: " + usuario.getNombre());
            Log.d("msg-test", "Correo: " + usuario.getCorreo());
        }
    });
```

Listeners y consumo de batería

Tenga en cuenta que aún si su aplicación se va a background, el **listener** sigue corriendo, por eso es una buena práctica desregistrar el listener cuando se va a onPause o onStop y volver a retomarlo en onResume.

Sin embargo, si desea escuchar en tiempo real las actualizaciones sobre su base de datos, entonces sirve dejar el listener activo.

Obtener el Key

Si se desea obtener el id o Key de cada registro, puede utilizar el método del documento **getId()**:

```
for(QueryDocumentSnapshot doc: snapshot){  
    Log.d("msg-test","ID: " + doc.getId());  
    UsuarioDto usuario = doc.toObject(UsuarioDto.class);  
}
```


Ordenamiento y filtrado

También puede ordenar y filtrar información, lo cual puede verificar en:

<https://firebase.google.com/docs/firestore/query-data/queries?hl=es-419>

Así mismo, también es posible guardar la información si el usuario está offline.

```
snapshotListener = db.collection("usuarios")  
    .orderBy("nombre", Query.Direction.ASCENDING)  
    .limit(3)  
    .addSnapshotListener((collection, error) => {
```

¿Preguntas?

Muchas gracias



Ingeniería de las
Telecomunicaciones